Laporan Proyek Berpikir Komputasional (WI1102)

Kelas K05



Disusun Oleh:

Jovan Nathaniel Permana (16026295)
Qamila Annur Firdausi (16026380)
Abira Mutia Salsabila (16026060)
Ezra Hadyan Wicaksono (16026055)

Fakultas FMIPA Institut Teknologi Bandung

Semester 1 Tahun 2026/2027

1 Dasar-dasar Berpikir Komputasional yang Digunakan

1.1 Algoritma Newton-Raphson

1.1.1 Dekomposisi

Algoritma Newton-Raphson sebenarnya adalah algoritma optimasi yang bersifat iteratif, dengan tujuan utama untuk mencari akar-akar/titik potong sebuah fungsi $f(x)$ dengan sumbu x. Dalam proyek ini, algoritma Newton-Raphson difokuskan untuk mencari akar-akar dari sebuah polinom dengan derajat tertentu, sesuai masukan dari pengguna (*user*). Melalui prinsip dekomposisi (pemecahan suatu masalah dengan skala besar menjadi skala yang lebih kecil), algoritma Newton-Raphson untuk mencari akar-akar sebuah fungsi polinom dapat dipecah menjadi beberapa tugas berikut.

1. Menginput derajat polinom (N), nilai koefisien-koefisiennya, serta nilai tebakan awal (x_0) dari user.
2. Untuk nilai tebakan awal $x = x_0$ yang diberikan, hitung nilai fungsi $f(x_0)$ beserta turunannya, yaitu $f'(x_0)$.
3. Nilai x diperbarui secara terus-menerus melalui prosedur iteratif dengan persamaan
$$x_{n+1} \leftarrow x_n - \frac{f(x_n)}{f'(x_n)} \tag{1}$$
hingga nilai *error* yang terjadi dalam penentuan akar menjadi tidak signifikan atau berada di bawah ambang batas tertentu.
4. Jika $f'(x) = 0$ pada suatu tahap tertentu atau perulangan pada langkah 3 tidak dapat selesai (*infinite loop*), maka proses perhitungan akar gagal dilakukan.

1.1.2 Pengenalan Pola

Algoritma Newton-Raphson memiliki pola yang mirip dengan algoritma optimasi, yaitu algoritma yang menekankan perhitungan matematika secara iteratif (berulang) untuk mencapai suatu tujuan. Dalam algoritma Newton-Raphson, persamaan (1) dijalankan secara berulang-ulang untuk mencapai suatu tujuan, yaitu akar persamaan polinom. Selama perulangan tersebut, nilai x akan terus diperbarui (*update*) hingga error dalam pencarian akar menjadi tidak signifikan atau lebih kecil dari ambang batas tertentu.

1.1.3 Abstraksi

Algoritma Newton-Raphson membutuhkan beberapa informasi penting, yaitu:

1. Derajat polinom N.
2. Koefisien-koefisien pada polinom.
3. Nilai tebakan awal x_0.

Dalam proyek ini, derajat polinom N disimpan di dalam variabel bertipe `int`, sementara nilai tebakan awal x_0 disimpan di dalam variabel bertipe `float`. Nilai-nilai koefisien pada polinom disimpan di dalam sebuah array `float` yang berukuran $N + 1$.

1.2 Algoritma Pinjam Buku

1.2.1 Dekomposisi

1.2.2 Pengenalan Pola

1.2.3 Abstraksi

1.3 Algoritma Labirin

1.3.1 Dekomposisi

1.3.2 Pengenalan Pola

1.3.3 Abstraksi

2 Lampiran

2.1 Lampiran 1: Kode Python Tanpa Menggunakan Subprogram

2.1.1 Algoritma Newton-Raphson

Kode 1: Kode Python algoritma Newton-Raphson tanpa subprogram

```
1 import time
2 """ Newton Raphson (tanpa subprogram) """
3
4 # program utama
5 # input
```

```
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = 0.0
    el = 0
    element = 1
    while el <= (N-1):
        val += coefs[el] * (x ** el)
        el += 1
    grad = 0.0
    while element <= (N-1):
        grad += element * coefs[element] * (x ** (element - 1))
        element += 1

    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f} Gradien: {grad:.6f}')
    delta = x_new - x
    if delta < 0:
        delta = -delta

```

```
    if val > 0:
        absval = val
    else:
        absval = -val

    if (delta < epsilon) and (absval < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.1.2 Algoritma Pinjam Buku

Kode 2: Kode Python algoritma Pinjam Buku tanpa subprogram

```
import time
"""Pinjam Buku (Tanpa Subprogram)"""
# program utama
# input
N = int(input("Masukkan jumlah orang: "))
matrix = []
umur = [0 for _ in range(N)]
umur_tersedia = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Masukkan isi matrix graph umur untuk koordinat ({x},{y}): '))
        matrix[y][x] = val
    print('================================')

```

```
for i in range(N):
    val_umur = int(input(f'Masukkan umur untuk index {i}: '))
    umur[i] = val_umur

pointer = int(input('Masukkan index tempat awal buku berada: '))
for i in range(N):
    if matrix[pointer][i] == 1:
        umur_tersedia += [umur[i]]

for i in range(len(umur_tersedia)):
    for j in range(i, len(umur_tersedia)):
        buffer = umur_tersedia[i]
        umur_tersedia[i] = umur_tersedia[j]
        umur_tersedia[j] = buffer
```

2.1.3 Algoritma Labirin

Kode 3: Kode Python algoritma Labirin tanpa subprogram

```
import time
"""labirin (tanpa Subprogram)"""
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Apakah koordinat ({x},{y}) bisa dilewati (1) atau tidak (0): '))
        matrix[y][x] = val
    print('================================')

# proses
start_time = time.perf_counter()
# inisialisasi pointer dan arahnya
status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
# cari titik awal di sisi atas
for i in range(N):
```

```
    if matrix[0][i] == 1:
        status_sisi[0] = [i,0]
        break
# cari titik awal di sisi bawah
for i in range(N):
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
avail_move = [0,0,0,0]
forbidden_move = [0,0,0,0]
clockwise = True
change_clockwise = 0
# jalankan iterasi
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    # cek arah yang tersedia
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
```

```
64          direction = [0,0,1,1]
65      elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
            :
66          direction = [1,0,0,1]
67      elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
68          direction = [0,1,1,1]
69      elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
            -1)):
70          direction = [1,1,0,1]
71      elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
72          direction = [1,1,1,0]
73      elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
            -1)):
74          direction = [1,0,1,1]
75      else:
76          direction = [1,1,1,1]
77
78      # cek apakah bisa dilewati
79      i = 0
80      while(i < 4):
81          if direction[i] == 1:
82              if (i == 0) and (forbidden_move[i] == 0):
83                  bukan_dinding = int(matrix[pointer
                        [1]-1][pointer[0]] == 1)
84                  avail_move[i] = bukan_dinding
85              elif (i == 1) and (forbidden_move[i] == 0):
86                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]+1] == 1)
87                  avail_move[i] = bukan_dinding
88              elif (i == 2) and (forbidden_move[i] == 0):
89                  bukan_dinding = int(matrix[pointer
                        [1]+1][pointer[0]] == 1)
90                  avail_move[i] = bukan_dinding
91              elif (i == 3) and (forbidden_move[i] == 0):
92                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]-1] == 1)
93                  avail_move[i] = bukan_dinding
94          else:
95              avail_move[i] = 0
96          i += 1
97      # Jika jalan buntu
98      if avail_move == [0,0,0,0]:
99          i = 0
```

```
100         while(i < 4):
101             if direction[i] == 1:
102                 if i == 0:
103                     bukan_dinding = int(matrix[pointer
                            [1]-1][pointer[0]] == 1)
104                     avail_move[i] = bukan_dinding
105                 elif i == 1:
106                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]+1] == 1)
107                     avail_move[i] = bukan_dinding
108                 elif i == 2:
109                     bukan_dinding = int(matrix[pointer
                            [1]+1][pointer[0]] == 1)
110                     avail_move[i] = bukan_dinding
111                 else:
112                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]-1] == 1)
113                     avail_move[i] = bukan_dinding
114             else:
115                 avail_move[i] = 0
116             i += 1
117 
118     # Pilih jalur
119     if clockwise:
120         j = 0
121         while (j < 4):
122             if avail_move[j] == 1:
123                 if j == 0:
124                     pointer[1] -= 1
125                     forbidden_move = [0,0,1,0]
126                     break
127                 elif j == 1:
128                     pointer[0] += 1
129                     forbidden_move = [0,0,0,1]
130                     break
131                 elif j == 2:
132                     pointer[1] += 1
133                     forbidden_move = [1,0,0,0]
134                     break
135                 else:
136                     pointer[0] -= 1
137                     forbidden_move = [0,1,0,0]
138                     break
```

```
            j += 1
    else:
        j = 3
        while (j >= 0):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j -= 1

    if (change_clockwise % (4*N)) == 0:
        clockwise = not clockwise

print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
   dilewati')
end_time = time.perf_counter()
execution_time = end_time - start_time
print('Labirin Diselesaikan!')
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2 Lampiran 2: Kode Python Dengan Menggunakan Subprogram

2.2.1 Algoritma Newton-Raphson

Kode 4: Kode Python algoritma Newton-Raphson dengan subprogram

```
import time
""" Newton Raphson (dengan subprogram) """

```

```
# definisi fungsi
def function(x, coefs):
    value = 0.0
    for i in range(len(coefs)):
        value += coefs[i] * (x**i)
    return value

def gradient(x, coefs):
    value = 0.0
    for i in range(1, len(coefs)):
        value += i * coefs[i] * (x**(i - 1))
    return value

def absol(x):
    value = (x >= 0) * x + (x < 0) * (-x)
    return value

# program utama
# input
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{
            i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = function(x, coefs)
    grad = gradient(x, coefs)
    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
```

```
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f}
       Gradien: {grad:.6f}')

    if (absol(x_new - x) < epsilon) and (absol(val) <
       epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k
           } iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2.2 Algoritma Pinjam Buku

Kode 5: Kode Python algoritma Pinjam Buku dengan subprogram

```
import time
"""Pinjam Buku (dengan Subprogram)"""
```

2.2.3 Algoritma Labirin

Kode 6: Kode Python algoritma Labirin dengan subprogram

```
import time
"""labirin (dengan Subprogram)"""
# definisi fungsi
def choose_dir(pointer):
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
```

```
10          direction = [0,0,1,1]
11      elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
            :
12          direction = [1,0,0,1]
13      elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
14          direction = [0,1,1,1]
15      elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
            -1)):
16          direction = [1,1,0,1]
17      elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
18          direction = [1,1,1,0]
19      elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
            -1)):
20          direction = [1,0,1,1]
21      else:
22          direction = [1,1,1,1]
23      return direction
24  # program utama
25  # input
26  N = int(input("Masukkan ukuran labirin: "))
27  matrix = []
28  for i in range(N):
29      col = [0]*N
30      matrix += [col]
31
32  for y in range(N):
33      for x in range(N):
34          val = int(input(f'Apakah koordinat ({x},{y})
                bisa dilewati (1) atau tidak (0): '))
35          matrix[y][x] = val
36      print('================================')
37
38  # proses
39  start_time = time.perf_counter()
40  # inisialisasi pointer dan arahnya
41  status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
42  # cari titik awal di sisi atas
43  for i in range(N):
44      if matrix[0][i] == 1:
45          status_sisi[0] = [i,0]
46          break
47  # cari titik awal di sisi bawah
48  for i in range(N):
```

```
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
avail_move = [0,0,0,0]
forbidden_move = [0,0,0,0]
clockwise = True
change_clockwise = 0
# jalankan iterasi
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    # cek arah yang tersedia
    direction = choose_dir(pointer)
    # cek apakah bisa dilewati
    i = 0
    while(i < 4):
        if direction[i] == 1:
            if (i == 0) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                   [1]-1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 1) and (forbidden_move[i] == 0):
```

```
90                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]+1] == 1)
91                  avail_move[i] = bukan_dinding
92              elif (i == 2) and (forbidden_move[i] == 0):
93                  bukan_dinding = int(matrix[pointer
                        [1]+1][pointer[0]] == 1)
94                  avail_move[i] = bukan_dinding
95              elif (i == 3) and (forbidden_move[i] == 0):
96                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]-1] == 1)
97                  avail_move[i] = bukan_dinding
98          else:
99              avail_move[i] = 0
100         i += 1
101     # Jika jalan buntu
102     if avail_move == [0,0,0,0]:
103         i = 0
104         while(i < 4):
105             if direction[i] == 1:
106                 if i == 0:
107                     bukan_dinding = int(matrix[pointer
                            [1]-1][pointer[0]] == 1)
108                     avail_move[i] = bukan_dinding
109                 elif i == 1:
110                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]+1] == 1)
111                     avail_move[i] = bukan_dinding
112                 elif i == 2:
113                     bukan_dinding = int(matrix[pointer
                            [1]+1][pointer[0]] == 1)
114                     avail_move[i] = bukan_dinding
115                 else:
116                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]-1] == 1)
117                     avail_move[i] = bukan_dinding
118             else:
119                 avail_move[i] = 0
120             i += 1
121
122     # Pilih jalur
123     if clockwise:
124         j = 0
125         while (j < 4):
```

```
126             if avail_move[j] == 1:
127                 if j == 0:
128                     pointer[1] -= 1
129                     forbidden_move = [0,0,1,0]
130                     break
131                 elif j == 1:
132                     pointer[0] += 1
133                     forbidden_move = [0,0,0,1]
134                     break
135                 elif j == 2:
136                     pointer[1] += 1
137                     forbidden_move = [1,0,0,0]
138                     break
139                 else:
140                     pointer[0] -= 1
141                     forbidden_move = [0,1,0,0]
142                     break
143             j += 1
144     else:
145         j = 3
146         while (j >= 0):
147             if avail_move[j] == 1:
148                 if j == 0:
149                     pointer[1] -= 1
150                     forbidden_move = [0,0,1,0]
151                     break
152                 elif j == 1:
153                     pointer[0] += 1
154                     forbidden_move = [0,0,0,1]
155                     break
156                 elif j == 2:
157                     pointer[1] += 1
158                     forbidden_move = [1,0,0,0]
159                     break
160                 else:
161                     pointer[0] -= 1
162                     forbidden_move = [0,1,0,0]
163                     break
164             j -= 1
165 
166     if (change_clockwise % (4*N)) == 0:
167         clockwise = not clockwise
168 
```

```
169 print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
        dilewati')
170 end_time = time.perf_counter()
171 execution_time = end_time - start_time
172 print('Labirin Diselesaikan!')
173 print(f"Waktu eksekusi: {execution_time:.6f} detik")
```